

Asynchronous processing with Redis Streams

Ingeniería de Sistemas

Universidad Austral — 2026

Synchronous vs asynchronous

Synchronous	Asynchronous
Every statement is waited for.	Execution isn't waited for; it will eventually run.
Execution is straightforward to follow.	Code executes in the background, alongside other code.
Linear code waits for function calls, including IO.	We can start parallel processes or perform IO without blocking.
Error handling is easy.	Error handling is harder: we don't know when execution will finish.

Asynchronous tools

- Java: `CompletableFuture`, `Future`
- Scala: `Future`
- JavaScript: native support and `Promise`
- Kotlin: coroutines
- Go: goroutines

Asynchronous use cases

- Read/write IO: IO is very slow compared to pure computation.
- Long-running jobs
- Batch jobs
- Chats
- UI
- Task queues

Our use case: changing linting/formatting rules

1. A user enables, disables or edits a linting rule.
2. Snippets are processed in the background.
3. The user sees which snippets have been revalidated and which are still pending validation.

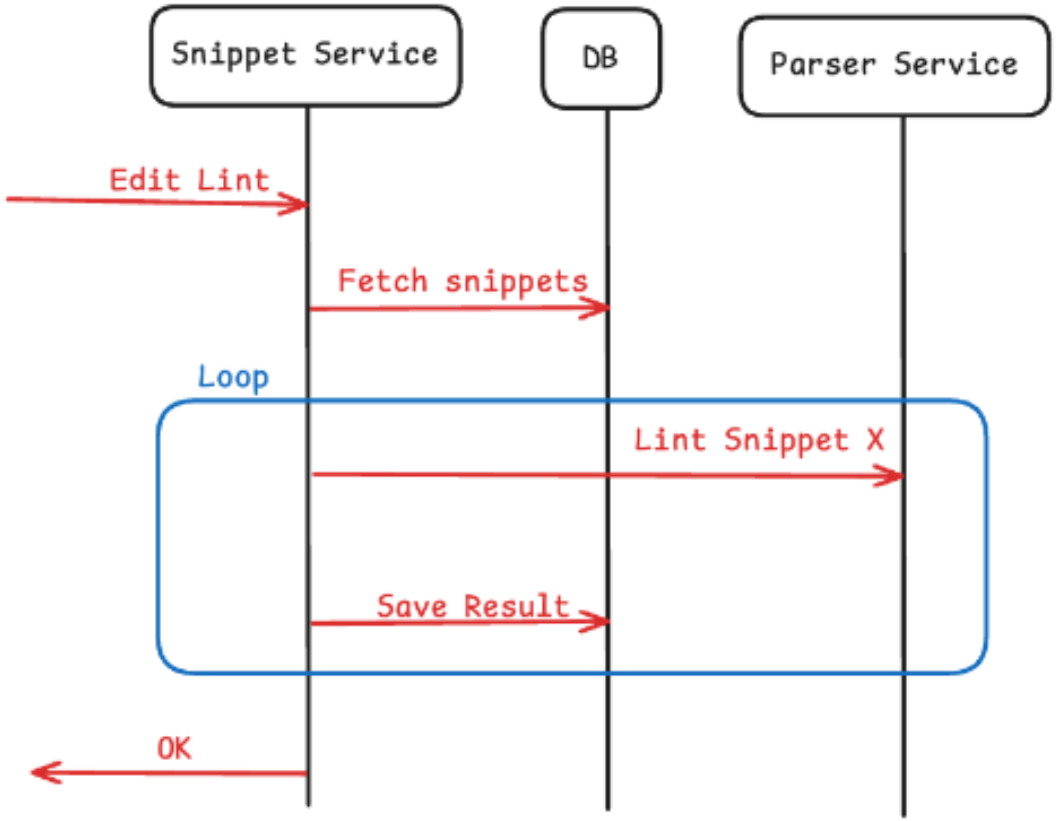
Approach #1

UNIVERSIDAD AUSTRAL

Query all snippets and lint one by one in the same request

1. The user edits a linting rule.
2. Snippet Service queries all snippets.
3. It executes lint one by one.
4. It saves the status of each lint.

Approach #1: execution flow



Approach #1: tradeoffs

UNIVERSIDAD AUSTRAL

Upsides

- Super easy to understand.

Downsides

- Slow.
- If execution is interrupted, do we resume or start again? Does it scale to thousands of snippets?
- Resource usage peaks.
- What happens to Parser Service?

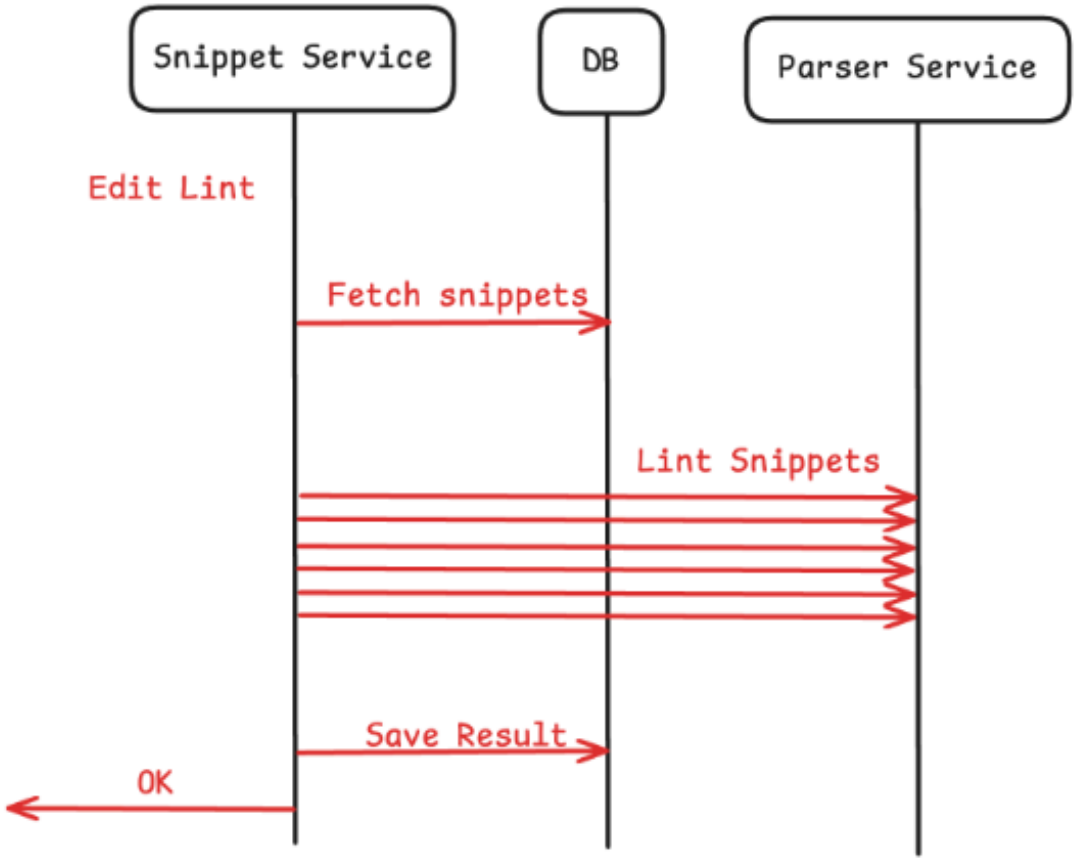
Approach #2

UNIVERSIDAD AUSTRAL

Query all snippets and lint them in parallel

1. The user edits a linting rule.
2. Snippet Service queries all snippets.
3. It lints all snippets in parallel.
4. It saves the reports.

Approach #2: execution flow



Approach #2: tradeoffs

Upsides

- Super easy to understand.
- Linting is fast because it runs at the same time.

Downsides

- What happens to Parser Service?
- If execution is interrupted, do we resume or start again? Does it scale to thousands of snippets?
- Resource usage peaks.

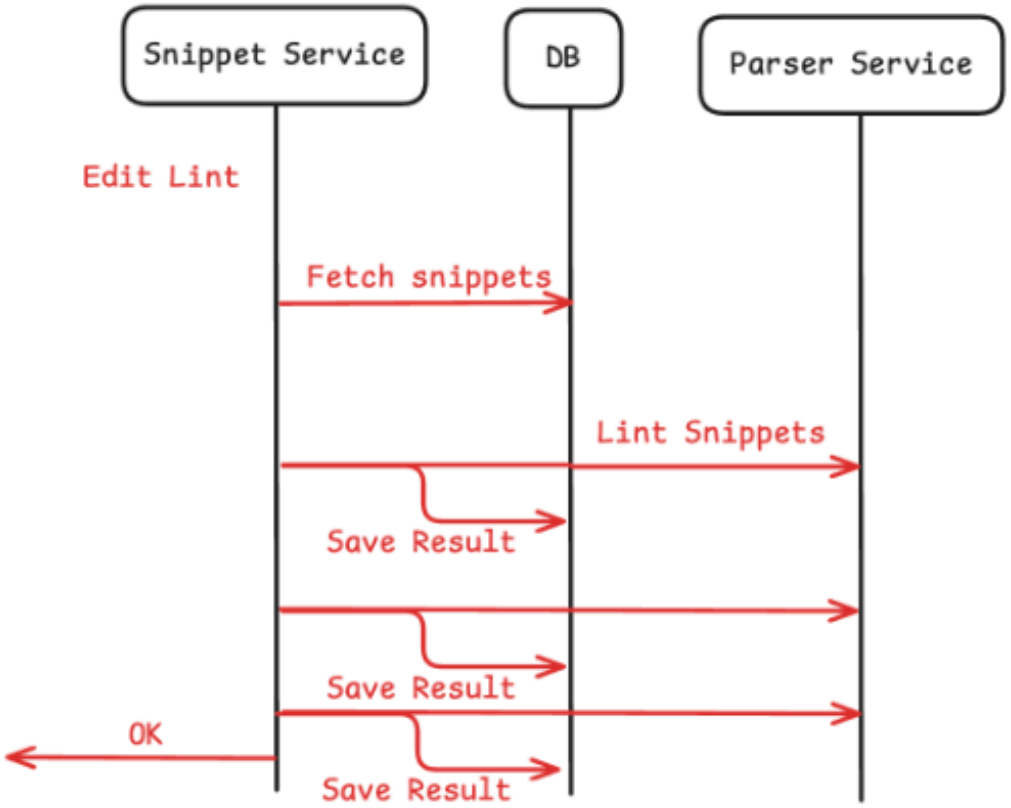
Approach #3

UNIVERSIDAD AUSTRAL

Query all snippets and lint them in parallel V2

1. The user edits a linting rule.
2. Snippet Service queries all snippets.
3. It lints all snippets in parallel.
4. Each time a lint finishes, it saves the result.

Approach #3: execution flow



Approach #3: tradeoffs

UNIVERSIDAD AUSTRAL

Upsides

- Linting is fast because it runs at the same time.
- We can have partial results.

Downsides

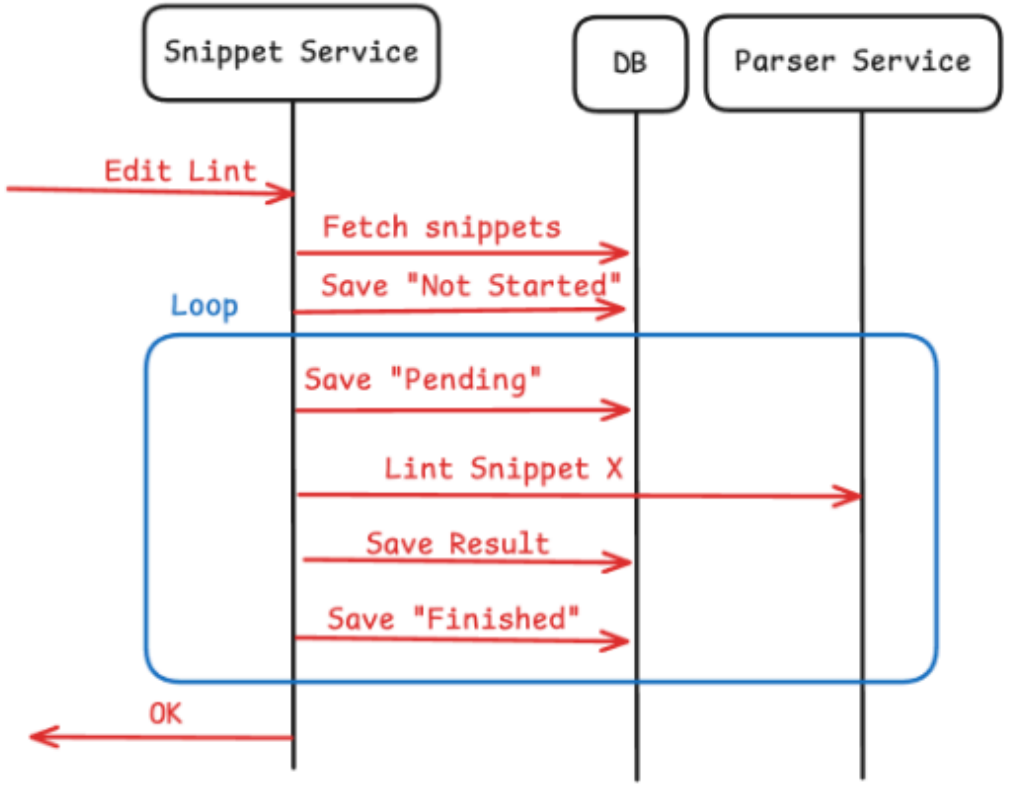
- What happens to Parser Service?
- If execution is interrupted, do we resume or start again? Does it scale to thousands of snippets?
- Resource usage peaks.

Approach #4

Query all snippets and save execution state

1. The user edits a linting rule.
2. Snippet Service queries all snippets.
3. It writes a row for each snippet in a Task table with status “Not Started”.
4. For each task: change status to “Pending”, run lint and store its report, then change status to “Finished”.

Approach #4: execution flow



Approach #4: tradeoffs

Upsides

- We can resume where we left off.

Downsides

- How do we resume?
- The solution is more complex.
- How do we know whether a “Pending” task is executing or was never marked “Finished”?

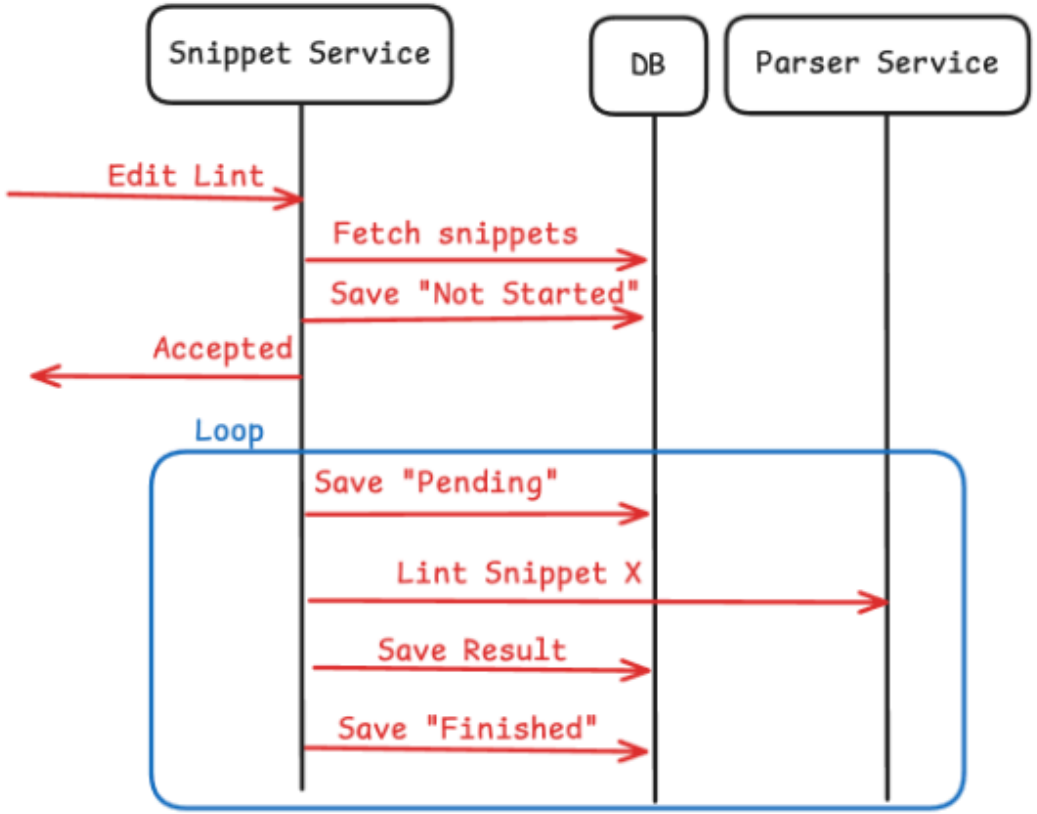
Approach #5

UNIVERSIDAD AUSTRAL

Query all snippets and save execution state V2

1. The user edits a linting rule.
2. Snippet Service queries all snippets.
3. It writes a row for each snippet in a Task table with status “Not Started”.
4. The request ends and returns “Accepted”.
5. In a side process, for each task: change status to “Pending”, run lint and store its report, then change status to “Finished”.

Approach #5: execution flow



Approach #5: tradeoffs

Upsides

- We can resume where we left off.
- We free request resources because the request has ended.

Downsides

- How do we resume?
- The solution is even more complex.
- How do we know whether a “Pending” task is executing or was never marked “Finished”?
- Error handling is more complex: the user may never know.

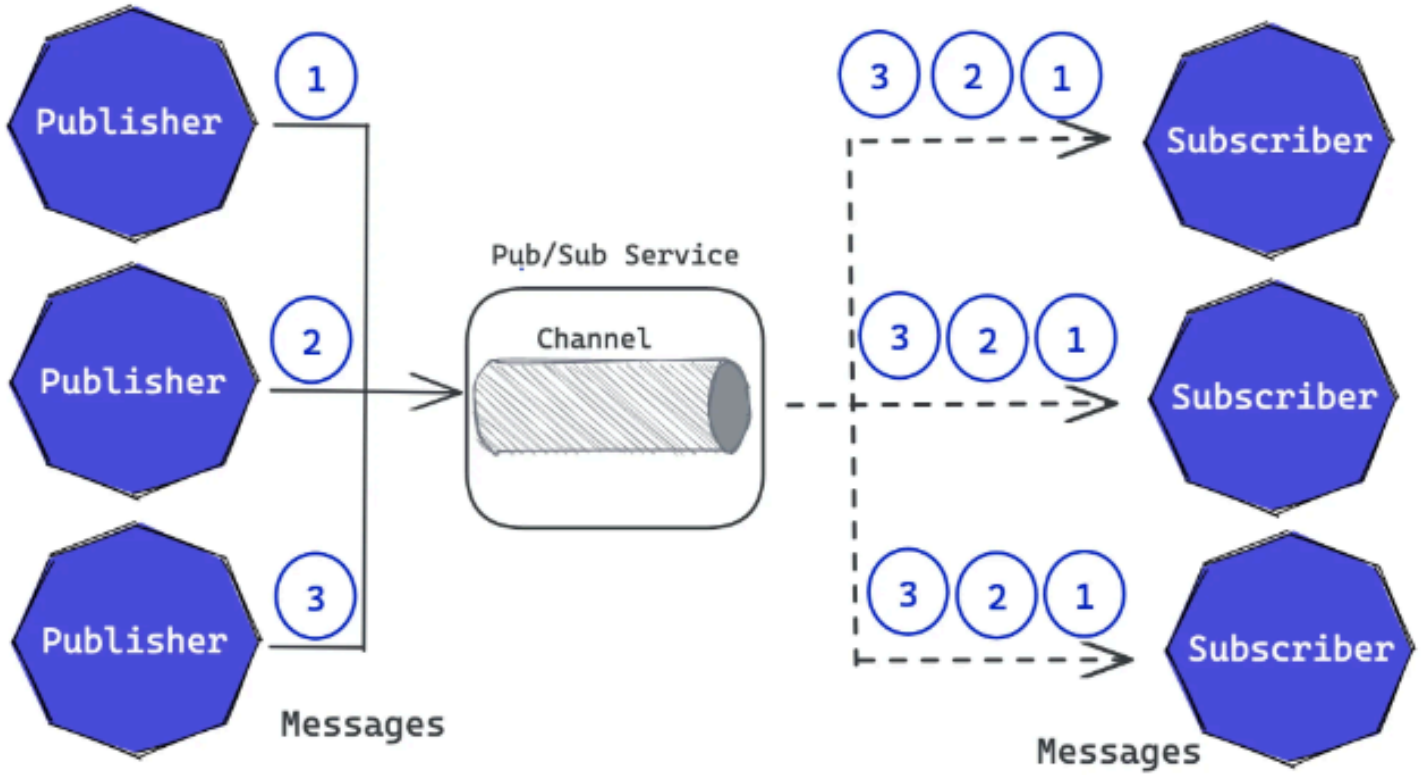
Dependency flow

What if we want to do more things when a user edits a lint rule, such as sending a metric or a notification?

To add behavior, we have to create more types of tasks and add code to Snippet Service. Not very *Open/Closed*.

Snippet Service is coupled to the effects of editing a linting rule.

Publisher–Consumer pattern



Publisher–Consumer pattern

- N producers
- M consumers
- Consumers don’t know producers, and vice versa.
- They communicate through messages.
- Consumers and producers are coupled only by the message schema.

Pub/Sub vs Streams

Main differences:

- Streams have history: a new consumer can process old messages.
- Messages can be retried without an ad hoc solution.
- Consumer groups
- Streams keep message order.
- Streams allow pacing.

Stream Providers

- Apache Kafka
- Apache Pulsar
- Redis
- AWS Kinesis
- Valkey